張皓翔 C14106080 期末專題

Canny 邊緣檢測

一、目錄

專題簡介

研究動機

演算法理論基礎

MATLAB 實作

實驗結果與分析

結論

參考資料

一、專題簡介

圖像處理技術在現代科技中扮演著至關重要的角色，廣泛應用於醫療診斷、機器視覺、監控系統等領域。邊緣檢測作為圖像處理中的基礎技術，旨在識別圖像中的物體邊界，是許多高級圖像處理任務的基礎。Canny 邊緣檢測演算法是由 John F. Canny 於 1986 年提出的，至今仍被視為最有效的邊緣檢測方法之一。

Canny 邊緣檢測器通過多階段處理來檢測圖像中的邊緣。其基本目標是通過抑制噪聲、計算梯度、進行非最大抑制和雙閾值檢測來精確地識別邊緣，從而達到實時和高效的邊緣檢測效果。該演算法不僅具有較高的檢測精度，還能在較大範圍內適應不同的圖像場景。

本專題旨在深入研究 Canny 邊緣檢測演算法的原理和實現方法，並通過 MATLAB 進行實作，探討其在實際應用中的效果和性能。希望通過本專題的研究，能夠對圖像處理技術有更深刻的理解，並為未來相關領域的研究和應用提供理論和技術支持。

二、研究動機

近期我對手機圖像處理功能的探索引發了對背景移除技術的強烈好奇心。在日常使用中，我發現手機照片在長按後會展示出一個令人驚奇的功能——自動去背景。這個發現讓我十分著迷，因為它不僅簡化了圖像編輯過程，還極大地擴展了個人創意的實現方式。例如，我可以迅速將朋友的照片去除背景，製作成多種有趣的貼圖，這些貼圖在我和朋友的簡訊互動中增添了許多樂趣。這種技術的神奇之處在於它如何精確地識別和分離前景和背景，這不僅涉及複雜的算法，還需要對圖像的細節進行深入分析。這驅使我想要深入了解其背後的原理和技術。

因此，我選擇將這一技術作為我的專題研究主題，希望通過學術研究來探索和理解背景移除技術的工作機制，包括它如何利用邊緣檢測算法如 Canny 來識別物體的邊界。這個專題不僅滿足了我對圖像處理技術的學術好奇心，也為我的技術技能帶來了實際應用的機會。在實作過程中，我將學習如何使用 MATLAB 來實現和優化這一過程。我希望透過專題的深入研究，能夠對圖像處理技術的未來發展趨勢和應用前景有更全面的理解。这不僅能增強我的專業技能，更能為將來在科技或軟件開發行業中的職業生涯奠定堅實的基礎。

三、演算法理論基礎

1.噪聲去除

由於邊緣檢測容易受到圖像噪聲的影響，首先使用高斯模糊濾波器對圖像進行平滑處理，這有助於減少圖像中的隨機噪聲。高斯模糊濾波器通過對像素及其鄰域內的像素進行加權平均來實現平滑效果。

2.梯度計算

接著，Canny 演算法會使用邊緣檢測運算子（如 Sobel 算子）來計算圖像中每個像素點的梯度強度和方向。梯度強度指的是在該點圖像亮度變化的速率，而梯度方向則是亮度變化最大方向的垂直方向。

3.非最大抑制

此步驟涉及細化邊緣過程，只保留梯度強度最大的邊緣作為真正的邊緣，這意味著演算法會消除那些非邊緣上的梯度最大值的點。通過在梯度方向上抑制非局部最大值的梯度強度，確保只有梯度最大的像素點被保留為邊緣。

4.雙閾值檢測和邊緣連接

最後，進行雙閾值檢測和邊緣連接。通過兩個閾值（一個高閾值和一個低閾值）來確定真實邊緣。高於高閾值的點被視為強邊緣，低於低閾值的點則被排

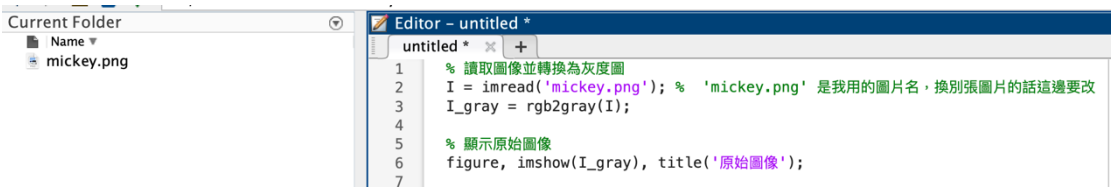
除。位於這兩個閾值之間的邊緣點則被視為弱邊緣點，其是否被保留取決於它們是否與強邊緣點相連。這樣可以有效地抑制噪聲，確保邊緣的連續性和完整性。

四、MATLAB 實作

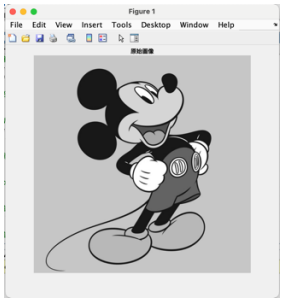
我會以 Mickey Mouse 的圖片為例，原圖為下圖，附圖一

⬅️附圖一

1.讀取和預處理圖像



首先，從文件中讀取圖像並將其轉換為灰度圖。灰度圖像僅包含亮度信息，簡化了後續的處理步驟。執行結果如下，附圖二



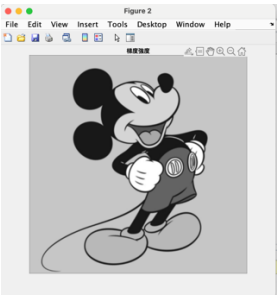
⬅️附圖二

*請注意，'mickey.png' 是我用的圖片名，換別張圖片的話這邊要改！

2.高斯模糊濾波器

```
% 應用高斯模糊濾波器進行噪聲去除
G = fspecial('gaussian', [5 5], 1.4);
I_smoothed = imfilter(I_gray, G, 'same');

% 顯示平滑後的圖像
figure, imshow(I_smoothed), title('高斯模糊後的圖像');
```

為了減少圖像中的噪聲，使用高斯模糊濾波器對圖像進行平滑處理。這一步的目的是通過模糊圖像來消除小的噪聲點，同時保留大部分邊緣信息。執行結果如下，附圖三

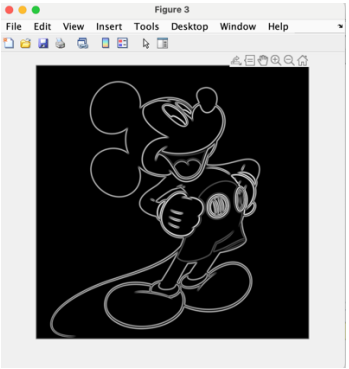


⬅️附圖三

3.梯度計算

```
% 使用 Sobel 算子計算水平和垂直方向的梯度
[Gx, Gy] = imgradientxy(I_smoothed, 'sobel');

% 計算梯度強度和方向
[gradient_magnitude, gradient_direction] = imgradient(Gx, Gy);

% 顯示梯度強度圖像
figure, imshow(gradient_magnitude, []), title('梯度強度');
```

使用 Sobel 算子計算圖像中每個像素點的水平和垂直梯度。梯度是一種向量，表示圖像亮度變化的方向和速率。水平和垂直梯度分別反映圖像在水平方向和垂直方向上的亮度變化。然後，根據水平和垂直梯度計算每個像素點的梯度強度和方向。執行結果如下，附圖四



⬅️附圖四

4.非最大抑制

```
% 非最大抑制的實現
[rows, cols] = size(gradient_magnitude);
non_max_suppressed = zeros(rows, cols);

% 對每一個像素進行處理
for i = 2:rows-1
    for j = 2:cols-1
        angle = gradient_direction(i, j);

        % 確保角度在 0 到 180 度之間
        if angle < 0
            angle = angle + 180;
        end

        % 根據梯度方向決定檢查的像素點
        if ((angle >= 0 && angle < 22.5) || (angle >= 157.5 && angle <= 180))
            check1 = gradient_magnitude(i, j + 1);
            check2 = gradient_magnitude(i, j - 1);
        elseif (angle >= 22.5 && angle < 67.5)
            check1 = gradient_magnitude(i + 1, j - 1);
            check2 = gradient_magnitude(i - 1, j + 1);
        elseif (angle >= 67.5 && angle < 112.5)
            check1 = gradient_magnitude(i + 1, j);
            check2 = gradient_magnitude(i - 1, j);
        elseif (angle >= 112.5 && angle < 157.5)
            check1 = gradient_magnitude(i - 1, j - 1);
            check2 = gradient_magnitude(i + 1, j + 1);
        end

        % 只有當該像素點的梯度強度大於相鄰的兩個點時才保留
        if (gradient_magnitude(i, j) >= check1 && gradient_magnitude(i, j) >= check2)
            non_max_suppressed(i, j) = gradient_magnitude(i, j);
        else
            non_max_suppressed(i, j) = 0;
        end
    end
end
```

```

% 顯示非最大抑制後的圖像
figure, imshow(non_max_suppressed, []), title('非最大抑制後的圖像');
```

為了細化邊緣，進行非最大抑制。這一步的目的是在邊緣的梯度方向上，抑制非局部最大值的梯度強度。具體實現中，對每個像素點，根據其梯度方向檢查其鄰域內的像素。如果該像素的梯度強度是其鄰域內最大的，則保留該像素的梯度強度，否則將其設為零。這樣可以確保只有梯度最大的像素點被保留為邊緣。執行結果如下，附圖五

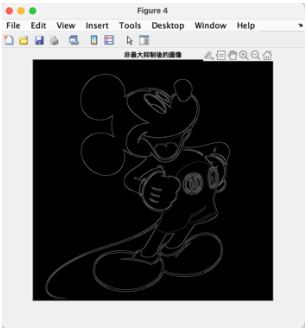


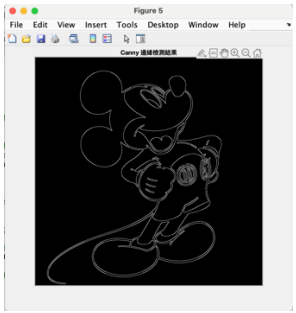
Figure 4
File Edit View Insert Tools Desktop Window Help
非最大抑制後的圖像

⬅️附圖五

5.雙閾值檢測和邊緣連接

```
% 雙閾值檢測
low_threshold = 0.1 * max(max(non_max_suppressed));
high_threshold = 0.3 * max(max(non_max_suppressed));
edges = zeros(rows, cols);

% 強邊緣
strong_edge = non_max_suppressed >= high_threshold;

% 弱邊緣
weak_edge = non_max_suppressed >= low_threshold & non_max_suppressed < high_threshold;

% 將強邊緣設為 1
edges(strong_edge) = 1;

% 邊緣連接，將與強邊緣連接的弱邊緣也設為 1
for i = 2:rows-1
    for j = 2:cols-1
        if weak_edge(i, j)
            if (strong_edge(i + 1, j - 1) || strong_edge(i + 1, j) || strong_edge(i + 1, j
                strong_edge(i, j - 1) || strong_edge(i, j + 1) || ...
                strong_edge(i - 1, j - 1) || strong_edge(i - 1, j) || strong_edge(i - 1, j
                edges(i, j) = 1;
            end
        end
    end
end

% 顯示最終邊緣檢測結果
figure, imshow(edges), title('Canny 邊緣檢測結果');
```

最後，進行雙閾值檢測和邊緣連接。首先，根據非最大抑制後的梯度強度設置高低閾值。高於高閾值的像素被認為是強邊緣，低於低閾值的像素被排除。介於兩者之間的像素被認為是弱邊緣，只有當它們與強邊緣連接時才被保留。這樣可以消除噪聲並確保邊緣的連續性。通過這一步，最終確定了圖像中的邊緣。執行結果如下，附圖六



⬅️附圖六

五、實驗結果與分析

1.原始圖像

首先，我們從文件中讀取了一張圖像並將其轉換為灰度圖。這張灰度圖像保留了原始圖像的亮度信息，但去除了顏色信息，簡化了後續的處理步驟。

2.高斯模糊濾波後的圖像

應用高斯模糊濾波器對灰度圖像進行平滑處理，減少了圖像中的噪聲。高斯模糊濾波器是一種線性濾波器，通過對像素及其鄰域內的像素進行加權平均來實現平滑效果。平滑後的圖像看起來更加柔和，細小的噪聲點被有效抑制，但主要的邊緣特徵仍然保留。

3.梯度強度圖像

使用 Sobel 算子計算圖像中每個像素點的水平和垂直梯度，並根據這些梯度計算每個像素點的梯度強度和方向。梯度強度圖像展示了每個像素點亮度變化的速率，亮度變化越劇烈的地方，其梯度強度越大。在梯度強度圖像中，邊緣處的像素點具有較高的梯度強度，顯示為亮色，而非邊緣處的像素點具有較低的梯度強度，顯示為暗色。

4.非最大抑制後的圖像

非最大抑制的目的是細化邊緣，保留梯度方向上的局部最大值，抑制非局部最大值的梯度強度。這一步驟消除了那些不是邊緣的梯度強度較大的像素點，保留了真正的邊緣。在非最大抑制後的圖像中，邊緣變得更加清晰和細緻，非邊緣處的噪聲被進一步減少。

5.雙閾值檢測和邊緣連接後的圖像

雙閾值檢測和邊緣連接是 Canny 邊緣檢測演算法的最後一步。這一步設置了高閾值和低閾值，強邊緣被直接保留，而弱邊緣只有在與強邊緣連接時才被保留。這樣可以有效地抑制噪聲，確保連續的邊緣。在最終的邊緣檢測結果圖像中，邊緣線條變得更加連貫和清晰，背景噪聲幾乎被完全消除。

6.結果分析

通過實驗結果可以看出，Canny 邊緣檢測演算法在邊緣檢測方面具有較高的精度和穩定性。每一步驟的處理對最終結果都有重要影響：

高斯模糊：有效減少噪聲，但保留主要邊緣特徵，為後續步驟打下良好基礎。

梯度計算：準確識別出圖像中亮度變化的方向和速率，確定潛在的邊緣。

非最大抑制：細化邊緣，保留真正的邊緣點，去除非局部最大值，減少誤檢。

雙閾值檢測和邊緣連接：進一步消除噪聲，確保邊緣的連續性和完整性。

六、結論

在這次專題研究中，我們成功地實現了 Canny 邊緣檢測演算法，並通過 MATLAB 程式碼進行了驗證和測試。從最初的圖像讀取和預處理，到最終的邊緣檢測結果，我們一步步探索了該演算法的各個關鍵步驟，並對每個步驟的效果進行了詳細的分析。

在進行實驗時，我們首先將圖像轉換為灰度圖，這一步簡化了後續的處理過程。接著，我們應用了高斯模糊濾波器，這個步驟有效地減少了圖像中的噪聲，同時保留了主要的邊緣特徵。通過這一步，我們為後續的梯度計算打下了良好的基礎。

隨後，我們使用 Sobel 算子計算了圖像中每個像素點的水平和垂直梯度，並根據這些梯度計算出了每個像素點的梯度強度和方向。在梯度強度圖像中，我們清晰地看到了亮度變化最劇烈的地方，這些地方通常就是邊緣所在的位置。

接下來，我們進行了非最大抑制，這一步的目的是細化邊緣，保留梯度方向上的局部最大值，並抑制其他非局部最大值的梯度強度。這樣，我們確保了只有真正的邊緣點被保留下來，消除了許多誤檢的邊緣。

最後，我們應用了雙閾值檢測和邊緣連接步驟。通過設置高閾值和低閾值，我們保留了強邊緣並進行了邊緣連接，確保了邊緣的連續性和完整性。在最終的邊緣檢測結果圖像中，我們看到邊緣線條變得更加連貫和清晰，背景噪聲幾乎被完全消除。

這次專題研究展示了 Canny 邊緣檢測演算法在圖像處理中的高效性和穩定性。每一步驟都對最終的邊緣檢測結果有著重要的影響，我們成功地在 MATLAB 中實現了這一演算法，並驗證了其在噪聲抑制和邊緣檢測方面的優越性能。

通過這次研究，我們不僅加深了對圖像處理技術的理解，還掌握了使用 MATLAB 進行圖像處理的實際操作技能。這些經驗和技能為我們未來在科技或軟件開發領域的職業生涯奠定了堅實的基礎。

在未來的工作中，我們可以進一步優化演算法的運行效率，將其應用於更複雜的圖像處理任務，如物體識別和圖像分割等，並結合其他先進的邊緣檢測方法，不斷提升檢測的精度和穩定性。這次專題研究為我們探索圖像處理技術的未來發展趨勢和應用前景提供了寶貴的理論和實踐支持。

七、參考資料

1. John F. Canny, "A Computational Approach to Edge Detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986.
2. Szeliski, R. "Computer Vision: Algorithms and Applications," Springer, 2010.
3. Gonzalez, R. C., & Woods, R. E. "Digital Image Processing," Prentice Hall, 2008.
4. MATLAB Documentation - MATLAB Edge Detection.
5. 邊緣檢測 - 維基百科.
6. humanking7, "Canny 邊緣檢測," CSDN 博客.
7. Mickey 圖片來源 - Spotify.